

Week 6 Structured Notes: Tower of Hanoi and Final Project

1. Week 6 Main Goal

The goal of Week 6 is to finish the recursion curriculum by learning a classic recursive problem called **Tower of Hanoi** and then combining all previous recursion programs into one final console project called **Recursion Playground**.

By the end of Week 6, you should be able to:

- Explain the Tower of Hanoi puzzle without code.
- Understand the three rods: source, auxiliary, and destination.
- Explain the two strict rules of Tower of Hanoi.
- Break the problem into smaller recursive steps.
- Write the recursive Tower of Hanoi function in C.
- Trace Tower of Hanoi for small inputs like 1, 2, 3, and 4 disks.
- Count the total number of moves.
- Explain why Tower of Hanoi has exponential time complexity.
- Build a menu-driven Recursion Playground project.
- Combine recursion concepts from all previous weeks into one program.

The main Week 6 idea is:

A big recursive problem can be solved by first solving a smaller version of the same problem.

Tower of Hanoi is one of the best examples of this idea.

2. Quick Revision Before Week 6

Before learning Tower of Hanoi, you should remember these ideas from earlier weeks.

2.1 Recursion

Recursion means a function calls itself.

Example:

```
void fun(int n) {  
    if (n > 0) {  
        fun(n - 1);  
    }  
}
```

```
}  
}
```

Here, `fun()` calls itself, so it is recursive.

2.2 Base Case

The base case is the condition that stops recursion.

Without a base case, recursion may continue forever and cause stack overflow.

Example:

```
if (n == 0) {  
    return;  
}
```

This says:

Stop when there is nothing left to process.

2.3 Recursive Case

The recursive case is the part where the function calls itself again with a smaller input.

Example:

```
fun(n - 1);
```

This reduces the problem from `n` to `n - 1`.

2.4 Calling Phase

The calling phase happens when recursive calls are going deeper.

Example:

```
fun(3)  
fun(2)  
fun(1)  
fun(0)
```

The function keeps calling itself until it reaches the base case.

2.5 Returning Phase

The returning phase happens after the base case is reached.

Then the function calls finish one by one and return to the previous call.

Example:

```
fun(0) returns  
fun(1) returns  
fun(2) returns  
fun(3) returns
```

2.6 Stack Memory

Each recursive call creates a new stack frame.

For example:

```
TOH(3)  
TOH(2)  
TOH(1)  
TOH(0)
```

Each call waits on the stack until the smaller call finishes.

2.7 Tree Recursion

Tree recursion happens when a function calls itself more than once.

Tower of Hanoi has two recursive calls:

```
TOH(n - 1, source, destination, auxiliary);  
TOH(n - 1, auxiliary, source, destination);
```

So Tower of Hanoi behaves like tree recursion.

3. Why Week 6 Is Important

Week 6 is important because it combines many recursion skills at once.

Tower of Hanoi uses:

- Base case
- Recursive case
- Calling phase
- Returning phase
- Stack behavior
- Tree recursion
- Recursive decomposition
- Exponential growth
- Manual tracing

The final project also tests whether you can organize many recursive functions into one clean menu-driven program.

This week is not only about writing code. It is about proving that you understand recursion deeply enough to explain, trace, analyze, and combine recursive programs.

4. What Is Tower of Hanoi?

Tower of Hanoi is a puzzle with:

- Disks
- Three rods
- Rules for moving disks

The rods are usually called:

```
A = Source rod  
B = Auxiliary rod  
C = Destination rod
```

Or:

```
Source      = where the disks start  
Auxiliary   = helper rod  
Destination = where the disks must go
```

The goal is:

Move all disks from the source rod to the destination rod.

The auxiliary rod is used as a temporary helper.

5. Tower of Hanoi Rules

There are two strict rules.

Rule 1: Move only one disk at a time

You cannot move two or more disks together.

Wrong idea:

Move disk 1 and disk 2 together

Correct idea:

Move one disk only

Rule 2: A bigger disk cannot be placed on a smaller disk

This is not allowed:

Big disk on top of small disk

This is allowed:

Small disk on top of big disk

These rules make the puzzle recursive.

6. Tower of Hanoi With 3 Disks

Suppose all disks start on rod A.

A = Source
B = Auxiliary
C = Destination

Goal:

Move all disks from A to C

For 3 disks, the big idea is:

Move 2 smaller disks from A to B
Move the largest disk from A to C
Move 2 smaller disks from B to C

Notice that to move 3 disks, we need to move 2 disks first.

That means:

Tower of Hanoi for 3 disks depends on Tower of Hanoi for 2 disks.

This is recursion.

7. The Core Recursive Logic

To move `n` disks from source to destination:

Step 1: Move $n - 1$ disks from source to auxiliary.
Step 2: Move the largest disk from source to destination.
Step 3: Move $n - 1$ disks from auxiliary to destination.

Using rods A, B, and C:

To move n disks from A to C:
1. Move $n - 1$ disks from A to B.
2. Move the largest disk from A to C.
3. Move $n - 1$ disks from B to C.

This is the most important Week 6 sentence.

If you understand this sentence, the code becomes much easier.

8. Why Move `n - 1` Disks First?

The largest disk is at the bottom.

You cannot move the largest disk until all smaller disks above it are removed.

So before moving the largest disk, you must move the top `n - 1` disks somewhere else.

That somewhere else is the auxiliary rod.

Simple idea:

The biggest disk is blocked by the smaller disks.
Move the smaller disks away first.
Then move the biggest disk.
Then place the smaller disks back on top.

9. Tower of Hanoi Function

Basic version:

```
void TOH(int n, int A, int B, int C) {  
    if (n > 0) {  
        TOH(n - 1, A, C, B);  
        printf("Move disk from %d to %d\n", A, C);  
        TOH(n - 1, B, A, C);  
    }  
}
```

Here:

n = number of disks
A = source rod
B = auxiliary rod
C = destination rod

A clearer version using better names:

```
void TOH(int n, int source, int auxiliary, int destination) {  
    if (n > 0) {  
        TOH(n - 1, source, destination, auxiliary);  
        printf("Move disk from %d to %d\n", source, destination);  
        TOH(n - 1, auxiliary, source, destination);  
    }  
}
```

```
}  
}
```

This version is easier to read because the parameter names explain their purpose.

10. Understanding the Parameters

In this function:

```
void TOH(int n, int source, int auxiliary, int destination)
```

The parameters mean:

Parameter	Meaning
n	Number of disks to move
source	Rod where the disks currently are
auxiliary	Helper rod
destination	Rod where the disks must go

Example call:

```
TOH(3, 1, 2, 3);
```

This means:

Move 3 disks from rod 1 to rod 3 using rod 2 as helper.

So:

```
1 = source  
2 = auxiliary  
3 = destination
```

11. Base Case in Tower of Hanoi

The function uses this condition:


```
if (n > 0)
```

This means the function only works when there is at least one disk.

When `n == 0`, the function does nothing.

So the hidden base case is:

If there are 0 disks, stop.

Another way to write it is:

```
if (n == 0) {  
    return;  
}
```

Then the function could be written like this:

```
void TOH(int n, int source, int auxiliary, int destination) {  
    if (n == 0) {  
        return;  
    }  
  
    TOH(n - 1, source, destination, auxiliary);  
    printf("Move disk from %d to %d\n", source, destination);  
    TOH(n - 1, auxiliary, source, destination);  
}
```

Both versions mean the same thing.

12. Recursive Case in Tower of Hanoi

The recursive case has three parts.

```
TOH(n - 1, source, destination, auxiliary);  
printf("Move disk from %d to %d\n", source, destination);  
TOH(n - 1, auxiliary, source, destination);
```

Part 1

```
TOH(n - 1, source, destination, auxiliary);
```

Move `n - 1` disks from source to auxiliary.

In this call, the destination rod becomes the helper.

Part 2

```
printf("Move disk from %d to %d\n", source, destination);
```

Move the largest disk from source to destination.

This is the actual move printed by the current function call.

Part 3

```
TOH(n - 1, auxiliary, source, destination);
```

Move the `n - 1` disks from auxiliary to destination.

Now the source rod becomes the helper.

13. Why the Rod Order Changes

This is one of the most confusing parts for beginners.

Look at this call:

```
TOH(n - 1, source, destination, auxiliary);
```

The goal here is not to move disks to the final destination yet.

The goal is to move the smaller disks out of the way.

So we move them:

```
from source to auxiliary
```

That is why the old `auxiliary` becomes the new `destination`.

Now look at the second recursive call:

```
TOH(n - 1, auxiliary, source, destination);
```

After the largest disk is moved, the smaller disks are on the auxiliary rod.

Now we need to move them:

```
from auxiliary to destination
```

So the old `source` becomes the helper.

Simple rule:

The meaning of each rod changes depending on the current smaller problem.

14. Full Tower of Hanoi Program

```
#include <stdio.h>

void TOH(int n, int source, int auxiliary, int destination) {
    if (n > 0) {
        TOH(n - 1, source, destination, auxiliary);

        printf("Move disk from %d to %d\n", source, destination);

        TOH(n - 1, auxiliary, source, destination);
    }
}

int main() {
    int n;

    printf("Enter number of disks: ");
    scanf("%d", &n);

    if (n < 0) {
        printf("Invalid input. Number of disks cannot be negative.\n");
    } else {
        TOH(n, 1, 2, 3);
    }
}
```

```
    }  
  
    return 0;  
}
```

15. Dry Run: TOH(1, A, B, C)

Function call:

TOH(1, A, B, C)

Since `n > 0`, the function runs.

Step 1

TOH(0, A, C, B)

Since `n == 0`, stop.

Step 2

Move disk from A to C

Step 3

TOH(0, B, A, C)

Since `n == 0`, stop.

Output

Move disk from A to C

Number of moves

1

Formula:

$$2^1 - 1 = 1$$

16. Dry Run: TOH(2, A, B, C)

Function call:

TOH(2, A, B, C)

To move 2 disks from A to C:

Move 1 disk from A to B
Move largest disk from A to C
Move 1 disk from B to C

Full trace

```
TOH(2, A, B, C)
  TOH(1, A, C, B)
    TOH(0, A, B, C) -> stop
    Move disk from A to B
    TOH(0, C, A, B) -> stop

    Move disk from A to C

  TOH(1, B, A, C)
    TOH(0, B, C, A) -> stop
    Move disk from B to C
    TOH(0, A, B, C) -> stop
```

Output

Move disk from A to B
Move disk from A to C
Move disk from B to C

Number of moves

3

Formula:

$$2^2 - 1 = 3$$

17. Dry Run: TOH(3, A, B, C)

Function call:

TOH(3, A, B, C)

To move 3 disks from A to C:

Move 2 disks from A to B
Move largest disk from A to C
Move 2 disks from B to C

Full output

Move disk from A to C
Move disk from A to B
Move disk from C to B
Move disk from A to C
Move disk from B to A
Move disk from B to C
Move disk from A to C

Number of moves

7

Formula:

$$2^3 - 1 = 7$$

18. Move Count Pattern

The number of moves follows this pattern:

Number of Disks	Total Moves	Formula
1	1	$2^1 - 1$
2	3	$2^2 - 1$
3	7	$2^3 - 1$
4	15	$2^4 - 1$
5	31	$2^5 - 1$

General formula:

$$\text{Total moves} = 2^n - 1$$

19. Why the Formula Is $2^n - 1$

Let $T(n)$ mean the number of moves needed for n disks.

To move n disks:

Move $n - 1$ disks first	-> $T(n - 1)$ moves
Move the largest disk	-> 1 move
Move $n - 1$ disks again	-> $T(n - 1)$ moves

So:

$$T(n) = T(n - 1) + 1 + T(n - 1)$$

That becomes:

$$T(n) = 2T(n - 1) + 1$$

Now expand:

```
T(1) = 1
T(2) = 2T(1) + 1 = 2(1) + 1 = 3
T(3) = 2T(2) + 1 = 2(3) + 1 = 7
T(4) = 2T(3) + 1 = 2(7) + 1 = 15
```

The pattern is:

```
T(n) = 2^n - 1
```

20. Time Complexity of Tower of Hanoi

Tower of Hanoi has time complexity:

```
O(2^n)
```

Why?

Because the number of moves grows like this:

```
1, 3, 7, 15, 31, 63, ...
```

This grows exponentially.

Another reason:

Each call creates two smaller recursive calls:

```
TOH(n - 1, source, destination, auxiliary);
TOH(n - 1, auxiliary, source, destination);
```

So the total work doubles many times.

This is similar to tree recursion.

21. Space Complexity of Tower of Hanoi

Tower of Hanoi has space complexity:

$O(n)$

Why?

Even though the total number of moves is exponential, the deepest recursive path is only:

```
TOH(n)
TOH(n - 1)
TOH(n - 2)
...
TOH(0)
```

So the maximum stack depth is proportional to n .

That means stack space is:

$O(n)$

Important difference:

Concept	Complexity
Total moves	$O(2^n)$
Maximum stack depth	$O(n)$

22. Why Tower of Hanoi Is Not Good for Large Inputs

Tower of Hanoi grows very fast.

For example:

```
n = 10 -> 2^10 - 1 = 1,023 moves
n = 20 -> 2^20 - 1 = 1,048,575 moves
n = 30 -> 2^30 - 1 = 1,073,741,823 moves
```

So even though the code is short, the output can become huge.

Simple warning:

Do not test Tower of Hanoi with large values unless you really want a huge amount of output.

For practice, use:

```
n = 1
n = 2
n = 3
n = 4
```

These are enough to understand the logic.

23. Tower of Hanoi as Recursive Decomposition

Recursive decomposition means breaking one big problem into smaller versions of the same problem.

Tower of Hanoi does this clearly.

Big problem:

```
Move n disks from A to C
```

Smaller problems:

```
Move n - 1 disks from A to B
Move n - 1 disks from B to C
```

The middle move is simple:

```
Move the largest disk from A to C
```

So the full structure is:

```
Smaller problem
Current direct work
Smaller problem
```

In code:

```
TOH(n - 1, source, destination, auxiliary);
printf("Move disk from %d to %d\n", source, destination);
TOH(n - 1, auxiliary, source, destination);
```

24. Tower of Hanoi Compared With Earlier Problems

Problem	Recursive Pattern	Time	Space
Sum	One recursive call	$O(n)$	$O(n)$
Factorial	One recursive call	$O(n)$	$O(n)$
Basic power	One recursive call	$O(n)$	$O(n)$
Fast power	One recursive call with half input	$O(\log n)$	$O(\log n)$
Fibonacci recursion	Two recursive calls	$O(2^n)$	$O(n)$
Tower of Hanoi	Two recursive calls plus one move	$O(2^n)$	$O(n)$

Tower of Hanoi is similar to Fibonacci recursion because both have more than one recursive call.

But there is an important difference:

- Fibonacci has repeated subproblems and can be improved with memoization.
- Tower of Hanoi must actually perform all moves, so the exponential number of moves is part of the puzzle.

25. Common Beginner Mistakes in Tower of Hanoi

Mistake 1: Memorizing the code without understanding the rods

Wrong habit:

```
Just memorize TOH(n - 1, A, C, B)
```

Correct habit:

```
Understand what source, auxiliary, and destination mean in each call.
```

Mistake 2: Forgetting the base case

Wrong:

```
void TOH(int n, int source, int auxiliary, int destination) {
    TOH(n - 1, source, destination, auxiliary);
    printf("Move disk from %d to %d\n", source, destination);
    TOH(n - 1, auxiliary, source, destination);
}
```

Problem:

There is no stopping condition.

Correct:

```
if (n > 0) {
    // recursive logic
}
```

Mistake 3: Thinking all rod names stay fixed

The rods change roles inside recursive calls.

A rod that is the destination in one call can become the auxiliary rod in another call.

Mistake 4: Testing with large `n`

Even `n = 20` creates over one million moves.

Use small inputs first.

Mistake 5: Confusing time complexity and space complexity

Tower of Hanoi has:

```
Time complexity =  $O(2^n)$ 
Space complexity =  $O(n)$ 
```

The total work is exponential, but the maximum stack depth is linear.

26. Beginner Mental Model

Think of Tower of Hanoi like this:

```
I cannot move the biggest disk until the smaller disks are out of the way.
```

So recursion says:

```
Move the smaller disks away.  
Move the biggest disk.  
Move the smaller disks back on top.
```

That is the whole puzzle.

Another simple version:

```
To move a tower, first move the top part.  
Then move the bottom disk.  
Then move the top part again.
```

27. How to Trace Tower of Hanoi

Use this method:

1. Write the function call.
2. Identify `n`, source, auxiliary, and destination.
3. Check the base case.
4. Write the first recursive call.
5. Write the printed move.
6. Write the second recursive call.
7. Keep expanding until `n == 0`.
8. Write the output in order.
9. Count the moves.
10. Compare the count with `2^n - 1`.

Example:

```
TOH(2, A, B, C)
```

Expected number of moves:

$$2^2 - 1 = 3$$

So your output should have 3 moves.

28. Week 6 Practice Questions

Question 1

What are the three rods in Tower of Hanoi?

Answer:

Source, auxiliary, and destination.

Question 2

What is the goal of Tower of Hanoi?

Answer:

Move all disks from the source rod to the destination rod using the auxiliary rod.

Question 3

What are the two rules?

Answer:

Move only one disk at a time.
Do not place a bigger disk on a smaller disk.

Question 4

What is the base case?

Answer:

When $n == 0$, stop.

Question 5

What is the recursive idea?

Answer:

```
Move n - 1 disks away.  
Move the largest disk.  
Move n - 1 disks onto the largest disk.
```

Question 6

How many moves are needed for 3 disks?

Answer:

```
2^3 - 1 = 7 moves
```

Question 7

What is the time complexity of Tower of Hanoi?

Answer:

```
O(2^n)
```

Reason:

```
The number of moves doubles as n grows.
```

Question 8

What is the space complexity of Tower of Hanoi?

Answer:

```
O(n)
```

Reason:

The deepest recursive call chain has length n .

Question 9

Why are there two recursive calls?

Answer:

One recursive call moves $n - 1$ disks away.
The second recursive call moves those $n - 1$ disks onto the destination rod.

Question 10

Why is the largest disk moved in the middle?

Answer:

The largest disk can move only after all smaller disks are removed from above it.

29. Final Project: Recursion Playground

The final project is a menu-driven console program.

It combines all major recursive functions from the curriculum.

The goal is to write one program where the user can choose which recursion example to run.

30. Recursion Playground Menu

The menu should look like this:

```
=====
Recursion Playground
=====
1. Print numbers descending
2. Print numbers ascending
3. Sum of N numbers
4. Factorial
```


5. Power
6. Fast Power
7. Fibonacci loop
8. Fibonacci recursion
9. Fibonacci memoization
10. nCr formula
11. nCr Pascal recursion
12. Tower of Hanoi
13. Exit

Each menu option should call a separate function.

This keeps the code organized.

31. Minimum Requirements for the Final Project

Your project should include:

31.1 Input handling

Ask the user for required input values.

Example:

```
Enter n:  
Enter base:  
Enter exponent:  
Enter number of disks:
```

31.2 Basic validation

Handle invalid values.

Examples:

```
Negative factorial input is invalid.  
Negative number of disks is invalid.  
r cannot be greater than n in nCr.
```

31.3 Correct output

Print the correct answer or correct moves.

Examples:

```
Factorial = 120  
Sum = 15  
Move disk from 1 to 3
```

31.4 Complexity notes

Add comments or print notes about complexity.

Example:

```
Time Complexity: O(n)  
Space Complexity: O(n)
```

31.5 Clear function names

Use names that explain the function.

Good names:

```
printDesc  
printAsc  
sumRecursive  
factorial  
power  
fastPower  
fibLoop  
fibRecursive  
fibMemo  
nCrFormula  
nCrPascal  
TOH
```

32. Better Version of the Final Project

A better version can include:

- Invalid input messages.
- Number of recursive calls.
- Complexity output after each operation.

- Loop comparison where useful.
- Clear comments explaining base cases.
- Separate functions for each menu option.
- A repeated menu until the user chooses exit.

Example:

```
Result = 120
Time Complexity: O(n)
Space Complexity: O(n)
```

This helps you connect coding with analysis.

33. Final Project Code Structure

A clean project can be organized like this:

```
#include <stdio.h>

// Week 1 functions
void printDesc(int n);
void printAsc(int n);

// Week 4 functions
int sum(int n);
int factorial(int n);
int power(int m, int n);
int fastPower(int m, int n);

// Week 5 functions
int fibLoop(int n);
int fibRecursive(int n);
int fibMemo(int n);
int nCrFormula(int n, int r);
int nCrPascal(int n, int r);

// Week 6 function
void TOH(int n, int source, int auxiliary, int destination);

int main() {
    int choice;

    do {
        // print menu
```

```
        // read choice
        // use switch-case
    } while (choice != 13);

    return 0;
}
```

This structure keeps the project readable.

34. Menu Logic With `switch`

A menu-driven program usually uses `switch`.

Basic idea:

```
switch (choice) {
    case 1:
        // call print descending
        break;

    case 2:
        // call print ascending
        break;

    case 12:
        // call Tower of Hanoi
        break;

    case 13:
        printf("Exiting program.\n");
        break;

    default:
        printf("Invalid choice.\n");
}
```

The `switch` checks the user's menu choice and runs the matching function.

35. Final Project Assessment Rubric

Your final project can be judged using these areas.

Criterion	Good Project Behavior
Correctness	Menu options work correctly for normal and edge inputs
Recursion understanding	Base cases and recursive cases are clear
Tracing ability	You can explain output step by step
Complexity analysis	Time and space complexity are correct
Code organization	Functions are clean, readable, and separated

The project should not look like copied code.

You should be able to explain every function.

36. Final Mastery Checklist

By the end of Week 6, you should be able to do these:

- Explain Tower of Hanoi in simple words.
- Name the source, auxiliary, and destination rods.
- State the two Tower of Hanoi rules.
- Explain the base case of Tower of Hanoi.
- Explain the recursive case of Tower of Hanoi.
- Trace `TOH(1)` manually.
- Trace `TOH(2)` manually.
- Trace `TOH(3)` manually.
- Count moves using $2^n - 1$.
- Explain why Tower of Hanoi is $O(2^n)$ time.
- Explain why Tower of Hanoi is $O(n)$ space.
- Explain why large inputs create huge output.
- Add Tower of Hanoi to a menu-driven program.
- Build the Recursion Playground project.
- Explain every menu option.
- Identify base cases in all final project functions.
- Identify recursive cases in all final project functions.
- Compare recursion with loops where suitable.
- Use memoization where repeated work happens.
- Trace recursive functions before running them.

37. Week 6 Final Summary

Week 6 teaches Tower of Hanoi and final project integration.

Tower of Hanoi is a recursive puzzle with three rods:

Source
Auxiliary
Destination

The goal is to move all disks from source to destination while following two rules:

Move one disk at a time.
Never place a bigger disk on a smaller disk.

The recursive logic is:

Move $n - 1$ disks from source to auxiliary.
Move the largest disk from source to destination.
Move $n - 1$ disks from auxiliary to destination.

The base case is:

If $n == 0$, stop.

The total number of moves is:

$2^n - 1$

So the time complexity is:

$O(2^n)$

The space complexity is:

$O(n)$

because the maximum recursive stack depth is n .

The final project is a menu-driven Recursion Playground that combines all previous topics:

Basic recursion
Stack understanding
Types of recursion
Sum
Factorial
Power
Fast power
Fibonacci
Memoization
nCr
Tower of Hanoi

The main Week 6 lesson is:

Recursion becomes powerful when you can break a big problem into smaller versions of the same problem.

38. Final Rule for Week 6

Every time you see Tower of Hanoi, ask these questions:

1. How many disks are there?
2. What is the source rod?
3. What is the auxiliary rod?
4. What is the destination rod?
5. What is the base case?
6. What are the two smaller recursive problems?
7. Where does the largest disk move?
8. How many total moves are needed?
9. What is the time complexity?
10. What is the space complexity?

If you can answer these questions, you understand Tower of Hanoi.

Do not memorize the function blindly.

Understand this sentence:

To move n disks from A to C:
move $n - 1$ disks from A to B,
move the largest disk from A to C,
then move $n - 1$ disks from B to C.

That sentence is the heart of Week 6.